Section F Tasks

Lab Task 1: Online Product Catalog

Scenario:

An e-commerce platform stores product entries by ProductID. The system must support price range queries and fast lookups for checkout and inventory management.

Task Requirements:

Each node should contain:
- ProductID (int, unique key)
- ProductName (string)
- Category (string)
- Price (float)

Core Functionalities:

- insertProduct(int id, string name, string category, float price)
- searchProduct(int id)
- deleteProduct(int id)
- displayProductsInOrder()
- getInorderIDs()

Advanced Operations:

- displayProductsInRange(float minPrice, float maxPrice)
- getMostExpensiveProduct()
- getLeastExpensiveProduct()

Note: Range queries should traverse using price filters after building candidate list if needed

Another Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

Google Test Cases:

```
TEST(ProductBSTTest, InsertAndSearchProduct) {
        ProductBST bst;
        bst.insertProduct(1, "Phone", "Electronics", 1200.0);
        bst.insertProduct(2, "TV", "Electronics", 2200.0);
        Product p = bst.searchProduct(2);
        EXPECT_EQ(p.ProductName, "TV"); // Expected correct product found
}

TEST(ProductBSTTest, DeleteProduct) {
        ProductBST bst;
        bst.insertProduct(5, "Laptop", "Tech", 1500.0);
        bst.deleteProduct(5);
   EXPECT_EQ(bst.searchProduct(5).ProductName, ""); // Expected deleted
}

TEST(ProductBSTTest, InorderSortedIDs) {
        ProductBST bst;
        bst.insertProduct(3, "A", "X", 50);
        bst.insertProduct(1, "B", "Y", 30);
        bst.insertProduct(2, "C", "Z", 40);
        vector<int> ino = bst.getInorderIDs();
        EXPECT_EQ(ino, (vector<int>{1,2,3})); // Expected sorted
}

TEST(ProductBSTTest, GetMostAndLeastExpensive) {
        ProductBST bst;
        bst.insertProduct(1, "A", "X", 50);
        bst.insertProduct(2, "B", "Y", 250);
        bst.insertProduct(3, "C", "Z", 10);
        Product max = bst.getMostExpensiveProduct();
        Product min = bst.getLeastExpensiveProduct();
        EXPECT_EQ(max.Price, 250); // Expected max price
   EXPECT_EQ(min.Price, 10);  // Expected min price
}

TEST(ProductBSTTest, ProductsInPriceRange) {
        ProductBST bst;
        bst.insertProduct(1, "P1", "Cat", 100);
        bst.insertProduct(2, "P2", "Cat", 200);
        bst.insertProduct(3, "P3", "Cat", 300);
        vector<Product> v = bst.displayProductsInRange(150, 250);
        EXPECT_EQ(v.size(), 1); // Expected only product with price 200
        EXPECT_EQ(v[0].ProductName, "P2");
}

TEST(ProductBSTTest, HeightAndCount) {
        ProductBST bst;
        bst.insertProduct(1, "A", "X", 10);
        bst.insertProduct(2, "B", "Y", 20);
```

```
    EXPECT_EQ(bst.getHeight(), 2); // Expected height after inserts
    EXPECT_EQ(bst.getTotalProducts(), 2); // Expected count
}

TEST(ProductBSTTest, DeleteNodeWithTwoChildren) {
    ProductBST bst;
    bst.insertProduct(50, "Root", "X", 100);
    bst.insertProduct(30, "L", "Y", 60);
    bst.insertProduct(70, "R", "Z", 120);
    bst.insertProduct(60, "M", "W", 80);
    bst.deleteProduct(50);
  EXPECT_EQ(bst.searchProduct(50).ProductName, ""); // Expected replaced root
}
```

Lab Task 2: File System Indexing Simulation Using Binary Search Trees

Scenario Overview

You are part of a development team building a **lightweight file indexing service** for a prototype operating system.
This system keeps track of files using a **Binary Search Tree (BST)** data structure. Each node in the BST represents a **file**, identified by a **unique File ID** (integer).

The file system frequently needs to:

- Locate a file quickly by its ID
- Track when each file was last accessed
- Remove outdated or unused files
- Generate reports such as “Which file was accessed least recently?”

Your task is to design and implement the BST to efficiently support these operations, while ensuring the BST remains correctly structured after updates.

Data Model

Each node in the BST should store:

FileID (int) → unique key used for BST ordering
FileName (string) → descriptive file name
LastAccessTime (int or datetime) → timestamp of the most recent access
FileSize (int) → size in KB/MB

Core Functionalities

Insertion of a New File

- Insert a new file record (FileID, FileName, FileSize, LastAccessTime).
- BST must maintain ordering based on `FileID`.
- Prevent duplicate FileIDs (if duplicate found, update LastAccessTime instead).

File Search

- Given a `FileID`, locate the corresponding file record.
- Return all details of the file.
- If not found, return an appropriate message.
- Update the file's `LastAccessTime` automatically after each successful search (simulating file access).

File Deletion

- Delete a file by `FileID`.
- The BST must remain structurally valid after deletion (handle leaf, single-child, and two-child cases).

Retrieve Least Recently Accessed File

- Implement a function `getLeastRecentlyAccessed()` that traverses the BST to find the file with the **oldest LastAccessTime**.

Display All Files in Ascending Order of FileID

- Perform an **in-order traversal** to print all file records sorted by FileID.

Display All Files Sorted by LastAccessTime

- Perform a **custom traversal** (not BST-ordered) to list files by ascending or descending `LastAccessTime`.

- This should demonstrate traversal beyond simple in-order patterns.

Extract IDs

- You will have to create a function that simply returns an int array containing the File IDs. This is needed for the test cases late

Test Cases

```
TEST(FileSystemBSTTest, InsertAndSearchBasic) {
    FileSystemBST fs;

    EXPECT_TRUE(fs.insert(105, "report.docx", 256, "10:15"));
    EXPECT_TRUE(fs.insert(110, "notes.txt", 32, "10:20"));
    EXPECT_TRUE(fs.insert(101, "config.sys", 5, "09:30"));

    // Duplicate insert should fail or update access time
    EXPECT_FALSE(fs.insert(105, "report.docx", 256, "10:25"));

    FileNode* found = fs.search(110);
    ASSERT_NE(found, nullptr);
    EXPECT_EQ(found->fileName, "notes.txt");
    EXPECT_EQ(found->fileSize, 32);

    EXPECT_EQ(fs.search(999), nullptr);  // nonexistent file
}
```

```
TEST(FileSystemBSTTest, InOrderTraversalSorted) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");

    auto inorder = extractIDs(fs.getInOrder());
    int expected[] = {101, 105, 110};
```

```
    EXPECT_EQ(inorder, expected);
}
```

```
TEST(FileSystemBSTTest, DeleteOperations) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");

    // Delete leaf
    EXPECT_TRUE(fs.deleteFile(110));
    // Delete one-child node
    EXPECT_TRUE(fs.deleteFile(101));
    // Delete root (two children)
    EXPECT_TRUE(fs.deleteFile(105));

    EXPECT_EQ(fs.search(105), nullptr);
}
```

```
TEST(FileSystemBSTTest, LeastRecentlyAccessedFile) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:10");

    FileNode* least = fs.getLeastRecentlyAccessed();
    ASSERT_NE(least, nullptr);
    EXPECT_EQ(least->fileID, 101);
}
```

```
TEST(FileSystemBSTTest, SortByAccessTime) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");
```

```
    auto sorted = extractIDs(fs.getFilesSortedByAccessTime());
    int expected[] = {101, 105, 110};

    ASSERT_EQ(sorted.size(), 3);
    EXPECT_TRUE(std::equal(sorted.begin(), sorted.end(), expected));
}
```

```
TEST(FileSystemBSTTest, HeightMeasurement) {
    FileSystemBST fs;
    fs.insert(10, "a.txt", 1, "09:00");
    fs.insert(5, "b.txt", 1, "09:01");
    fs.insert(15, "c.txt", 1, "09:02");
    fs.insert(2, "d.txt", 1, "09:03");
    fs.insert(7, "e.txt", 1, "09:04");

    EXPECT_EQ(fs.getHeight(), 3);
}
```